Python DA Capstone Project
Marketing Campaign Performance Insights

NAME : UMAYAL . K

SUBMISSION DATE : 10 - 11 - 2026

1) Load the Dataset:

```
import pandas as pd

url = "https://raw.githubusercontent.com/ArchanaInsights/Datasets/main/marketing_campaign.csv"
df = pd.read_csv(url)

print(df.shape)

df.head()
```

```
(22029, 16)
```

	Campaign_ID	Company	Campaign_Type	Target_Audience	Duration	Channel_Used	Conversion_Rate	Acquisition_Cost	ROI	Location
0	1	TechCorp	Email	Women 25-34	30 days	Facebook	5.294194	9344	62.94	Houston
1	2	Innovate Industries	Influencer	Women 35-44	45 days	Google Ads	3.326375	8783	10.67	Washington, D.C.

Activate Windows
Go to Settings to activate Windo

2) Descriptive Analysis:

Basic Structure:

➢Print the first few rows of the dataset to get an overview of the data.

(22029, 16)

[1]:

	Campaign_ID	Company	Campaign_Type	Target_Audience	Duration	Channel_Used	Conversion_Rate	Acquisition_Cost	ROI	Location
0	1	TechCorp	Email	Women 25-34	30 days	Facebook	5.294194	9344	62.94	Houston
1	2	Innovate Industries	Influencer	Women 35-44	45 days	Google Ads	3.326375	8783	10.67	Washington, D.C.
2	3	NexGen Systems	Social Media	Women 25-34	45 days	Instagram	4.056375	9111	73.20	Miami
3	4	Innovate Industries	Email	Women 25-34	45 days	Instagram	4.496375	7420	60.92	Seattle
4	5	Data Tech Solutions	Influencer	Men 25-34	30 days	Google Ads	4.405930	2146	138.82	Chicago

Activate Windows
Go to Settings to activate Windo

Language	Clicks	Impressions	Engagement_Score	Customer_Segment	Date
English	3045	67836	5	Tech Enthusiasts	01-01-2023
German	1944	66361	4	Foodies	01-01-2023
Spanish	3156	86240	8	Fashionistas	01-01-2023
Spanish	2388	58251	6	Foodies	01-01-2023
English	1025	34407	5	Tech Enthusiasts	01-01-2023

Activate Windows
Go to Settings to activate Windows.

➢Obtain the number of rows and columns in the dataset.

```
# Number of rows and columns
print(df.shape)
```

```
(22029, 16)
```

- The `shape` attribute was used to determine the dimensions of the dataset.
- It returns a tuple containing the number of rows and columns.
- This helps understand the size of the dataset before performing further analysis.

➢Get a concise summary of the dataset, including the data types and non-null values.

```
# Display dataset summary and non-null values
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 22029 entries, 0 to 22028
Data columns (total 16 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Campaign_ID       22029 non-null  int64
 1   Company           22029 non-null  str
 2   Campaign_Type     22029 non-null  str
 3   Target_Audience   22029 non-null  str
 4   Duration          22029 non-null  str
 5   Channel_Used      22029 non-null  str
 6   Conversion_Rate   22029 non-null  float64
 7   Acquisition_Cost  22029 non-null  int64
 8   ROI               22029 non-null  float64
 9   Location          22029 non-null  str
 10  Language          22029 non-null  str
 11  Clicks            22029 non-null  int64
 12  Impressions       22029 non-null  int64
 13  Engagement_Score  22029 non-null  int64
```

Activate Windows
Go to Settings to activate Windows.

```
 11  Clicks            22029 non-null  int64
 12  Impressions       22029 non-null  int64
 13  Engagement_Score  22029 non-null  int64
 14  Customer_Segment  22029 non-null  str
 15  Date              22029 non-null  str
dtypes: float64(2), int64(5), str(9)
memory usage: 4.5 MB
```

- The `info()` function was used to obtain a concise summary of the dataset.
- It provides information about the total number of entries, column names, non-null values, data types of each column, and memory usage.
- This helps identify missing values and understand the structure of the dataset before further analysis.

➢Generate descriptive statistics for numerical columns:

```
[4]: # Generate descriptive statistics
     df.describe()
```

[4]:

	Campaign_ID	Conversion_Rate	Acquisition_Cost	ROI	Clicks	Impressions	Engagement_Score
count	22029.000000	22029.000000	22029.000000	22029.000000	22029.000000	22029.000000	22029.000000
mean	11015.000000	4.757232	5522.740842	182.863648	2223.807572	50610.402787	6.582323
std	6359.368876	0.960393	2597.666260	301.619721	1394.166380	28542.979123	1.458804
min	1.000000	2.015723	1000.000000	-98.300000	30.000000	1001.000000	4.000000
25%	5508.000000	4.130705	3286.000000	-4.080000	1067.000000	25804.000000	5.000000
50%	11015.000000	4.761527	5525.000000	93.650000	2088.000000	50858.000000	7.000000
75%	16522.000000	5.429335	7766.000000	247.310000	3212.000000	75165.000000	8.000000
max	22029.000000	7.469907	9999.000000	3109.790000	6887.000000	99999.000000	9.000000

- The `describe()` method was used to generate descriptive statistics for the numerical columns in the dataset.
- It provides key statistical measures such as count, mean, standard deviation, minimum value, quartiles, and maximum value.
- These statistics help understand the central tendency, spread, and distribution of the numerical data.

Data Exploration:

➢Print the number of unique Campaign_ID values in the dataset.

```
# Number of unique Campaign_ID values
print("Number of unique Campaign_IDs:", df['Campaign_ID'].nunique())
```

```
Number of unique Campaign_IDs: 22029
```

- The `nunique()` function was used on the `Campaign_ID` column to determine the number of distinct campaign identifiers present in the dataset.
- This helps identify how many unique marketing campaigns are included in the analysis.

➢List the unique values of the Location and Customer_Segment columns.

```
# Unique values in Customer_Segment
print("\nUnique Customer Segments:")
print(df['Customer_Segment'].unique())
```

```
Unique Locations:
<ArrowStringArray>
[         'Houston', 'Washington, D.C.',            'Miami',
          'Seattle',          'Chicago',      'Los Angeles',
          'Atlanta',           'Dallas',         'New York',
    'San Francisco']
Length: 10, dtype: str

Unique Customer Segments:
<ArrowStringArray>
[   'Tech Enthusiasts',             'Foodies',        'Fashionistas',
 'Outdoor Adventurers',   'Health & Wellness']
Length: 5, dtype: str
```

- The `unique()` function was used to identify all distinct values in the `Location` and `Customer_Segment` columns.
- This helps understand the different geographical regions and customer categories represented in the dataset.
- The `nunique()` function was used to determine the number of unique values in each column.

➢Count the occurrences of each category in the Campaign_Type and **Channel_Used and columns.**

```
# Count occurrences of each Campaign_Type
print("Campaign Type Counts:")
print(df['Campaign_Type'].value_counts())

# Count occurrences of each Channel_Used
print("\nChannel Used Counts:")
print(df['Channel_Used'].value_counts())
```

```
Campaign Type Counts:
Campaign_Type
Display         4450
Search          4441
Social Media    4412
Email           4388
Influencer      4338
Name: count, dtype: int64
```

```
Channel Used Counts:
Channel_Used
Facebook      3742
Google Ads    3694
Website       3688
Instagram     3649
YouTube       3632
Email         3624
Name: count, dtype: int64
```

- The `value_counts()` function was used to count the frequency of each category in the `Campaign_Type` and `Channel_Used` columns.

3) Exploratory Data Analysis (EDA) and Visualization:

Campaign Performance:

➢Plot a scatter plot to visualize the relationship between Acquisition_Cost and **ROI.**

```
# 3) Exploratory Data Analysis (EDA) and Visualization

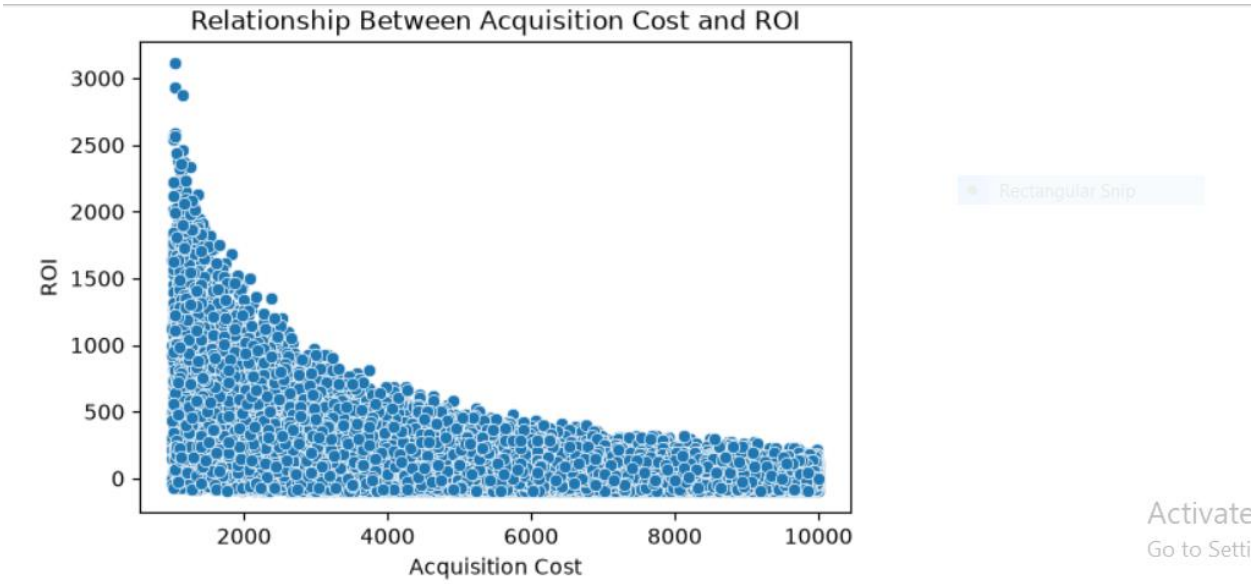
import matplotlib.pyplot as plt
import seaborn as sns

# Create scatter plot
plt.figure(figsize=(6, 4))
sns.scatterplot(data=df, x='Acquisition_Cost', y='ROI')

# Add title and labels
plt.title('Relationship Between Acquisition Cost and ROI')
plt.xlabel('Acquisition Cost')
plt.ylabel('ROI')

# Display plot
plt.show()
```

Activate Windows
Go to Settings to activate Windows



Activate
Go to Setti

- A scatter plot was used to visualize the relationship between **Acquisition_Cost** and **ROI**. Each point represents a campaign.

- This visualization helps identify trends, correlations, clusters, and potential outliers.
- If the points generally trend upward, it suggests that higher acquisition costs are associated with higher ROI; if they trend downward, it suggests an inverse relationship.

➢Create a bar chart to visualize the average Conversion_Rate for different **Channel_Used, categorized by Campaign_Type.**

[9]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Set figure size
plt.figure(figsize=(6, 4))

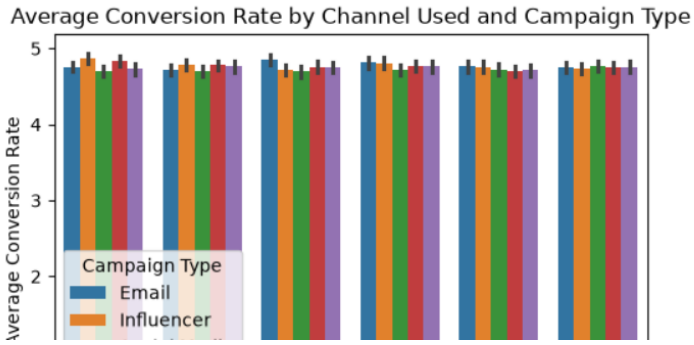
# Create bar chart
sns.barplot(
    data=df,
    x='Channel_Used',
    y='Conversion_Rate',
    hue='Campaign_Type',
    estimator='mean'
)

# Add title and labels
plt.title('Average Conversion Rate by Channel Used and Campaign Type')
plt.xlabel('Channel Used')
plt.ylabel('Average Conversion Rate')
```

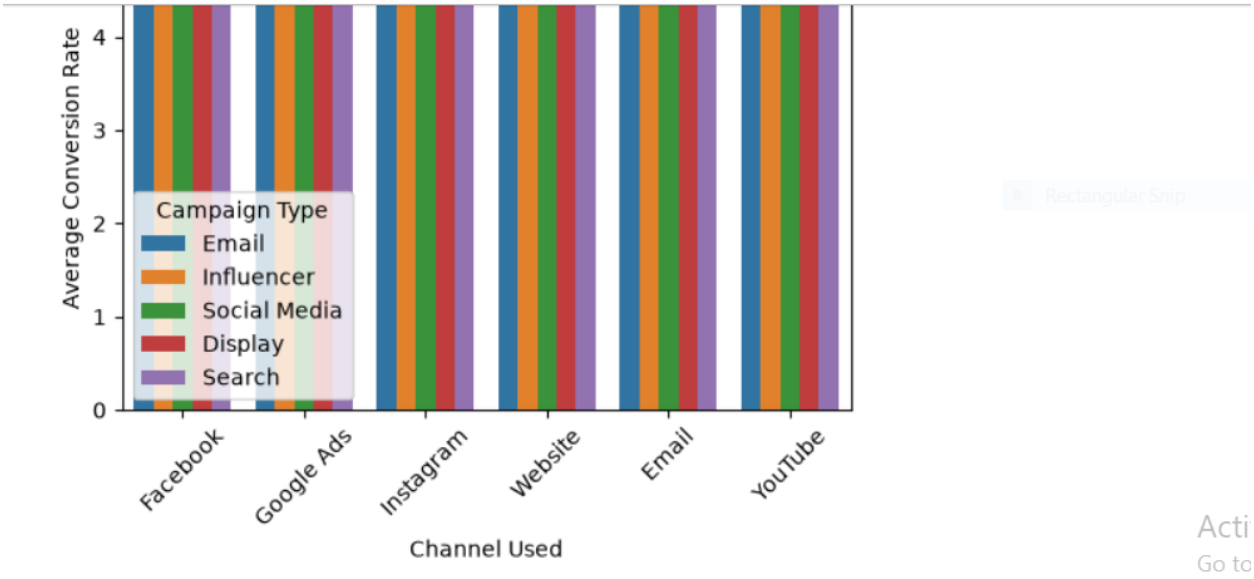
Activate Wind
Go to Settings to

```
# Rotate labels if needed
plt.xticks(rotation=45)

# Show legend and plot
plt.legend(title='Campaign Type')
plt.show()
```



Activate Windows
Go to Settings to activate Wind



- A grouped bar chart was created to compare the average **Conversion Rate** across different **marketing channels** (`Channel_Used`) and **campaign types** (`Campaign_Type`).
- The bars represent the mean conversion rate for each combination of channel and campaign type, enabling the identification of the most effective marketing strategies and channels.

➢Visualize the distribution of Engagement_Score across different Campaign_Type using a box plot.

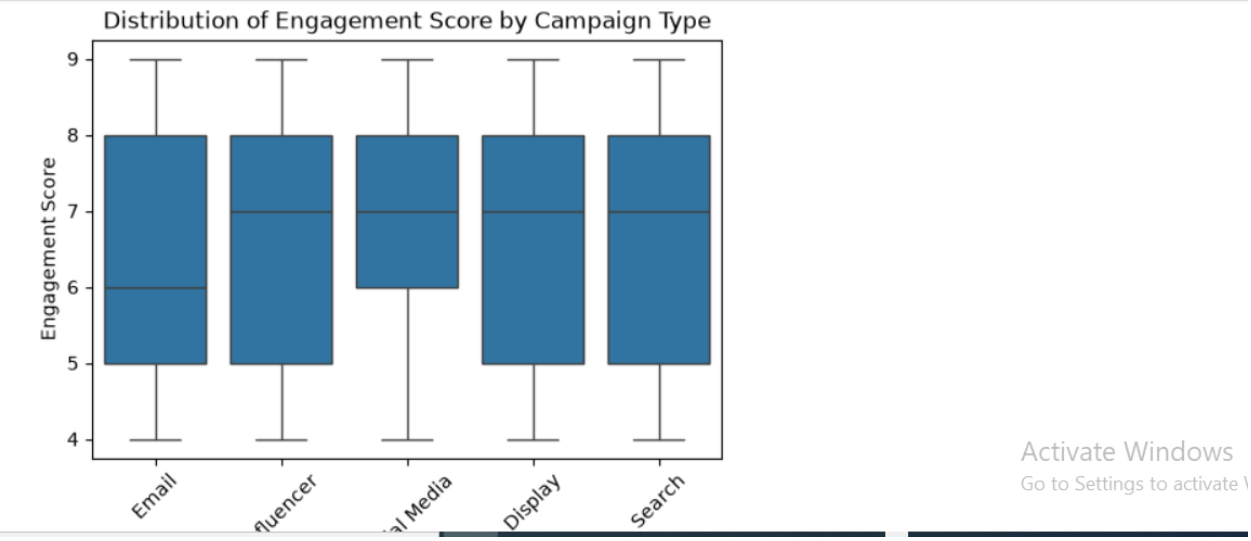
```
# Set figure size
plt.figure(figsize=(6, 4))

# Create box plot
sns.boxplot(
    data=df,
    x='Campaign_Type',
    y='Engagement_Score'
)

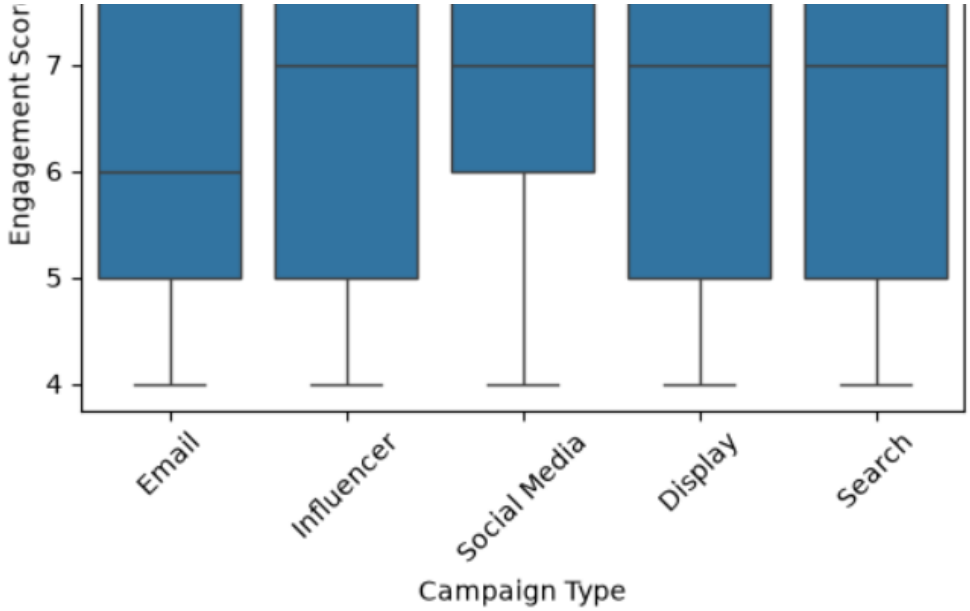
# Add title and labels
plt.title('Distribution of Engagement Score by Campaign Type')
plt.xlabel('Campaign Type')
plt.ylabel('Engagement Score')

# Rotate labels if needed
plt.xticks(rotation=45)

# Display plot
plt.show()
```



Activate Windows
Go to Settings to activate



- A box plot was used to visualize the distribution of **Engagement_Score** across different **Campaign_Type** categories.
- The box plot displays the median, quartiles, spread of the data, and potential outliers for each campaign type.
- This helps compare engagement performance and identify variations in customer engagement among different campaign strategies.

➢Analyze the average ROI by Company using a bar chart to compare the profitability **of campaigns conducted by different companies.**

```python
# Set figure size
plt.figure(figsize=(6, 4))

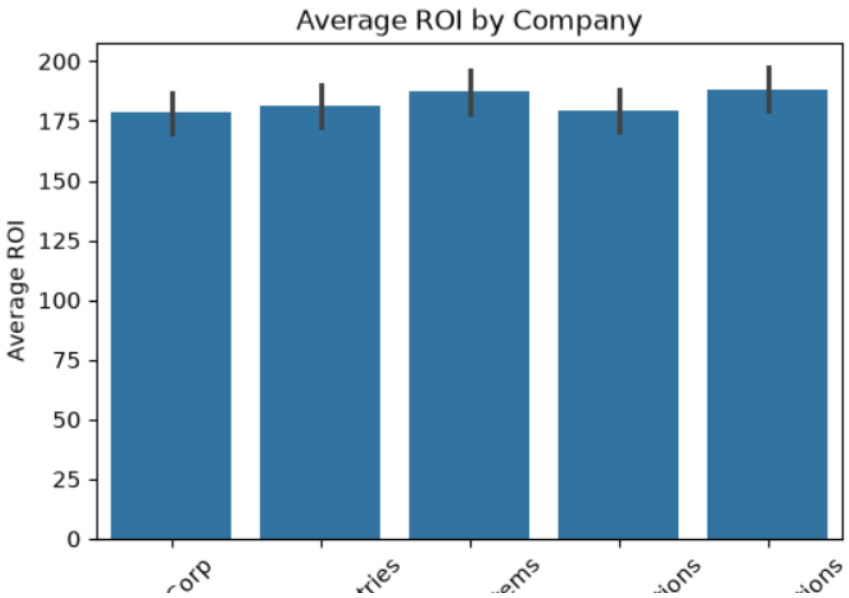
# Create bar chart
sns.barplot(
    data=df,
    x='Company',
    y='ROI',
    estimator='mean'
)

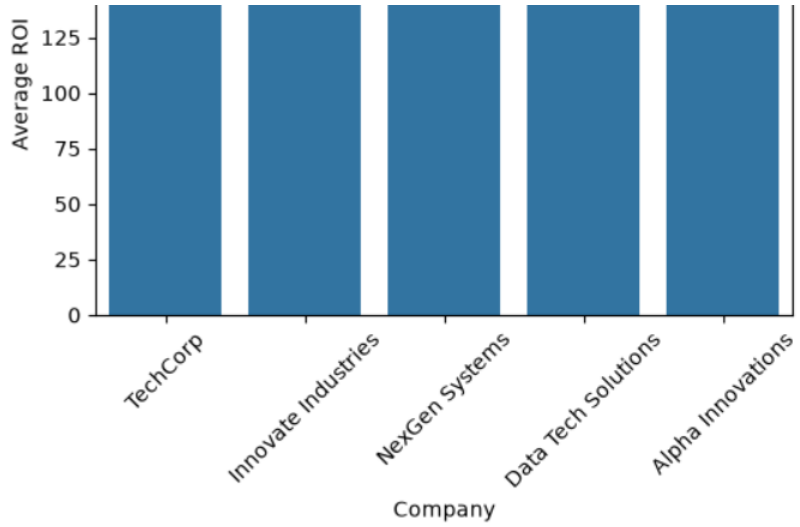
# Add title and labels
plt.title('Average ROI by Company')
plt.xlabel('Company')
plt.ylabel('Average ROI')

# Rotate company names if needed
plt.xticks(rotation=45)

# Display plot
plt.show()
```

Activate Windows
Go to Settings to activate Windows.





Activate W
Go to Settings

- A bar chart was used to compare the average **Return on Investment (ROI)** across different companies.
- The average ROI for each company was calculated and visualized to identify which companies achieved the highest profitability from their marketing campaigns.
- Companies with higher average ROI demonstrate more effective campaign performance and better returns on marketing investments.

➢Examine the correlation between Engagement_Score and Conversion_Rate **using a heatmap.**

[16]:
```
import matplotlib.pyplot as plt
import seaborn as sns

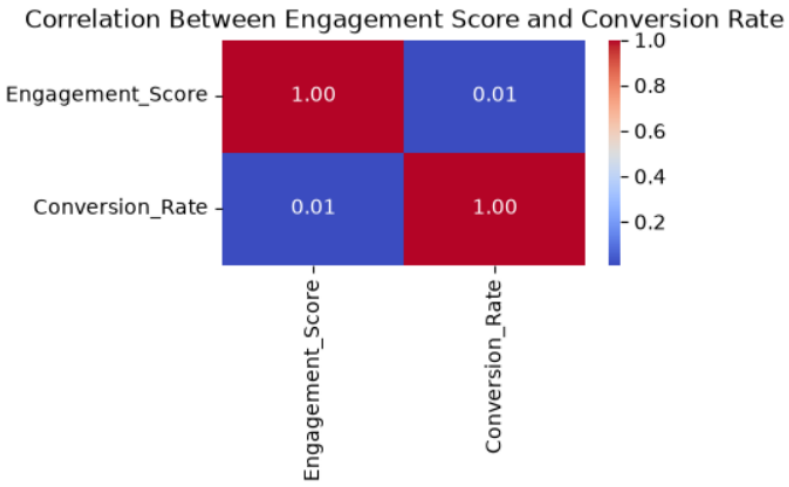
# Select the columns and calculate correlation
corr_matrix = df[['Engagement_Score', 'Conversion_Rate']].corr()

# Create heatmap
plt.figure(figsize=(4, 2))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

# Add title
plt.title('Correlation Between Engagement Score and Conversion Rate')

# Display heatmap
plt.show()
```

Activate Windows
Go to Settings to activate Windows.



Activate Winc
Go to Settings to a

- A correlation heatmap was used to examine the relationship between **Engagement_Score** and **Conversion_Rate**. The correlation coefficient ranges from -1 to +1:
- **+1** indicates a strong positive correlation.
- **-1** indicates a strong negative correlation.
- **0** indicates no linear relationship.
- The heatmap visually represents the strength and direction of the correlation, helping determine whether higher engagement levels are associated with higher conversion rates.

Customer Segmentation:

➢Create a count plot to visualize the distribution of Target_Audience.

```
#Customer Segmentation:

import matplotlib.pyplot as plt
import seaborn as sns

# Set figure size
plt.figure(figsize=(4, 2))

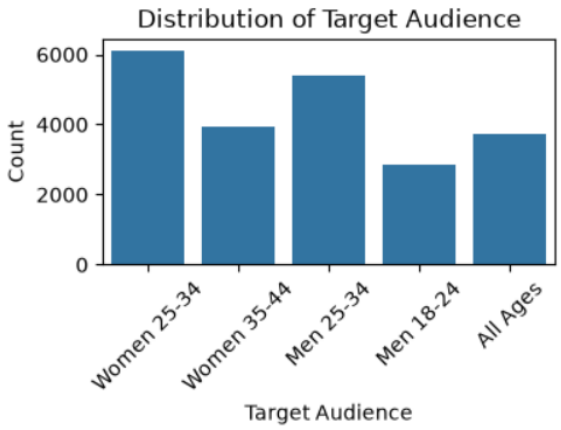
# Create count plot
sns.countplot(data=df, x='Target_Audience')

# Add title and labels
plt.title('Distribution of Target Audience')
plt.xlabel('Target Audience')
plt.ylabel('Count')

# Rotate labels if needed
plt.xticks(rotation=45)

# Display plot
plt.show()
```

Activate Windows
Go to Settings to activate Windows.



Rectangular Snip
Activate Wind

- A count plot was used to visualize the distribution of the **Target_Audience** categories in the dataset.
- The height of each bar represents the number of campaigns targeting a specific audience group.
- This visualization helps identify the most frequently targeted audience segments and provides insights into marketing focus areas.

➢Identify which Customer_Segment has the highest Conversion_Rate for each **Language using a bar chart.**

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate average Conversion Rate by Language and Customer Segment
avg_conversion = df.groupby(['Language', 'Customer_Segment'])['Conversion_Rate'].mean().reset_index()

# Get the Customer Segment with the highest Conversion Rate for each Language
top_segments = avg_conversion.loc[
    avg_conversion.groupby('Language')['Conversion_Rate'].idxmax()
]

# Display the results
print(top_segments)

# Create bar chart
plt.figure(figsize=(6, 4))
sns.barplot(
    data=top_segments,
    x='Language',
    y='Conversion_Rate',
```

Rectangular Snip
Activate Windows
Go to Settings to activate Windows.

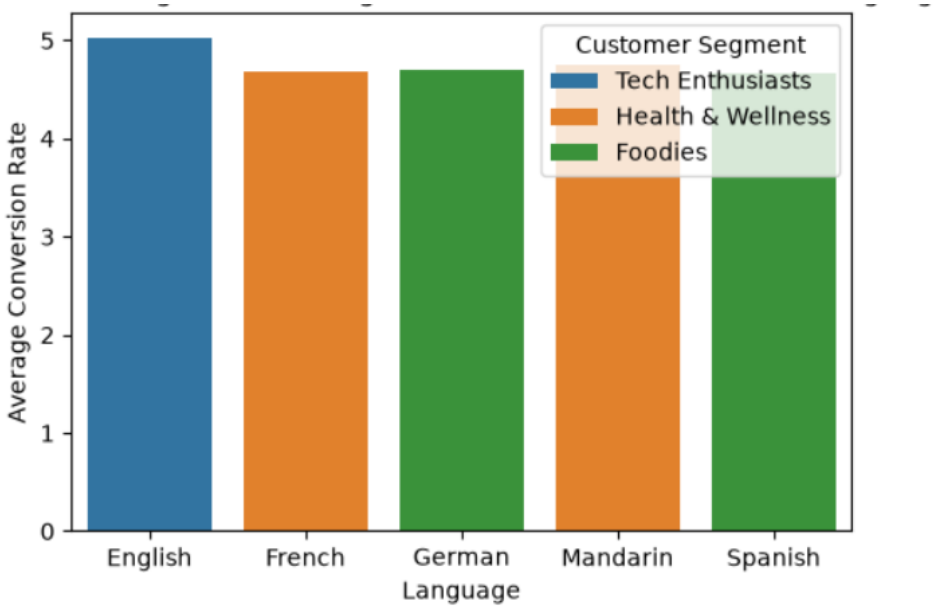
```
    hue='Customer_Segment'
)

plt.title('Customer Segment with Highest Conversion Rate for Each Language')
plt.xlabel('Language')
plt.ylabel('Average Conversion Rate')
plt.legend(title='Customer Segment')
plt.show()
```

	Language	Customer_Segment	Conversion_Rate
4	English	Tech Enthusiasts	5.022731
7	French	Health & Wellness	4.689704
11	German	Foodies	4.705430
17	Mandarin	Health & Wellness	4.756125
21	Spanish	Foodies	4.663610



Activate Windows
Go to Settings to activate W



- The data was grouped by **Language** and **Customer_Segment**, and the average **Conversion_Rate** was calculated for each combination.
- For each language, the customer segment with the highest average conversion rate was identified and visualized using a bar chart.
- This analysis helps determine which customer segment responds most effectively to marketing campaigns in different languages, providing valuable insights for targeted marketing strategies.

➢Visualize the distribution of Acquisition_Cost across each Customer_Segment, **categorized by Channel_Used, using a box plot.**

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Set figure size
plt.figure(figsize=(6, 4))

# Create box plot
sns.boxplot(
    data=df,
    x='Customer_Segment',
    y='Acquisition_Cost',
    hue='Channel_Used'
)

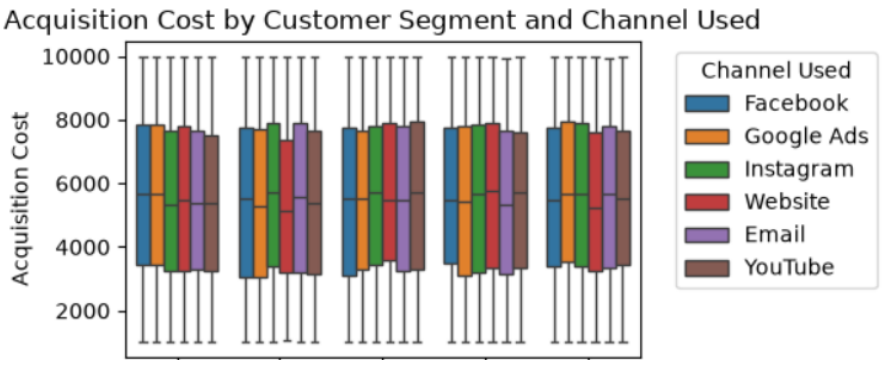
# Add title and labels
plt.title('Acquisition Cost by Customer Segment and Channel Used')
plt.xlabel('Customer Segment')
plt.ylabel('Acquisition Cost')

# Rotate labels if needed
plt.xticks(rotation=45)
```

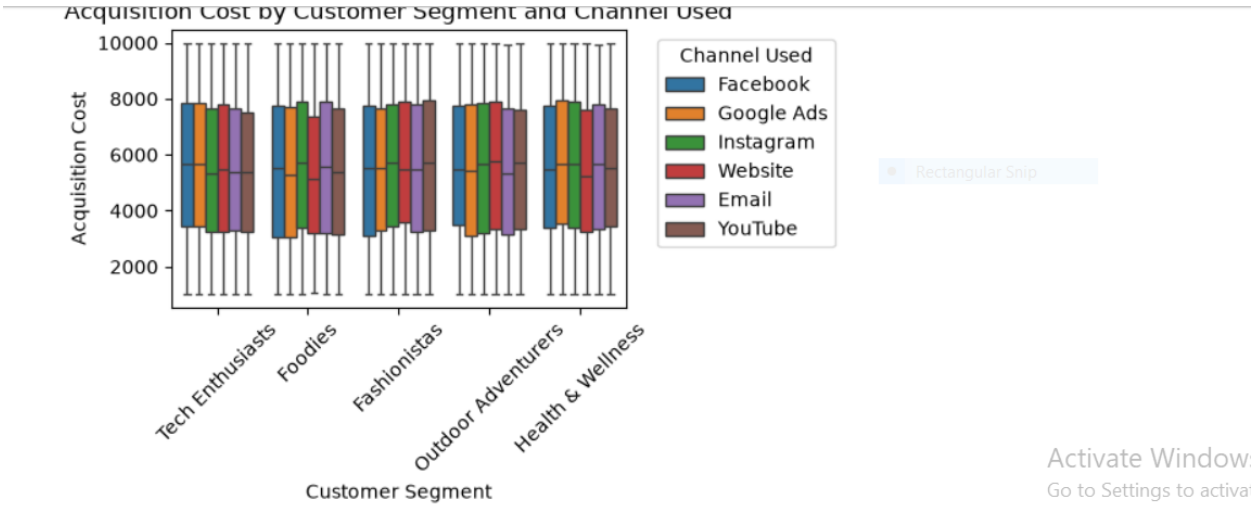
Activate Windows
Go to Settings to activate Windows.

```python
# Show legend
plt.legend(title='Channel Used', bbox_to_anchor=(1.05, 1), loc='upper left')

# Display plot
plt.tight_layout()
plt.show()
```



Activa
Go to Se



Activate Window
Go to Settings to activa

- A box plot was used to visualize the distribution of **Acquisition_Cost** across different **Customer_Segment** categories while distinguishing campaigns by **Channel_Used**.
- The plot shows the median, quartiles, spread, and potential outliers for acquisition costs within each segment and channel combination.
- This analysis helps identify differences in customer acquisition spending patterns across segments and marketing channels.

➢Analyze average Conversion_Rate by Language using a bar chart to compare **the effectiveness of campaigns conducted in different languages.**

```
# Set figure size
plt.figure(figsize=(6, 4))

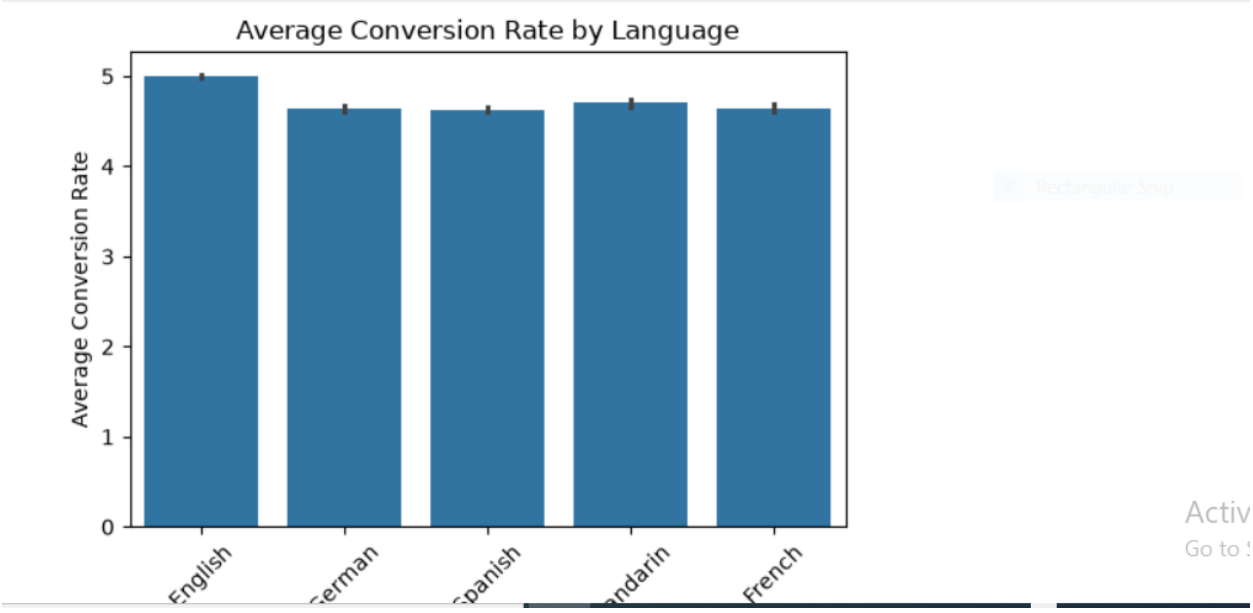
# Create bar chart
sns.barplot(
    data=df,
    x='Language',
    y='Conversion_Rate',
    estimator='mean'
)

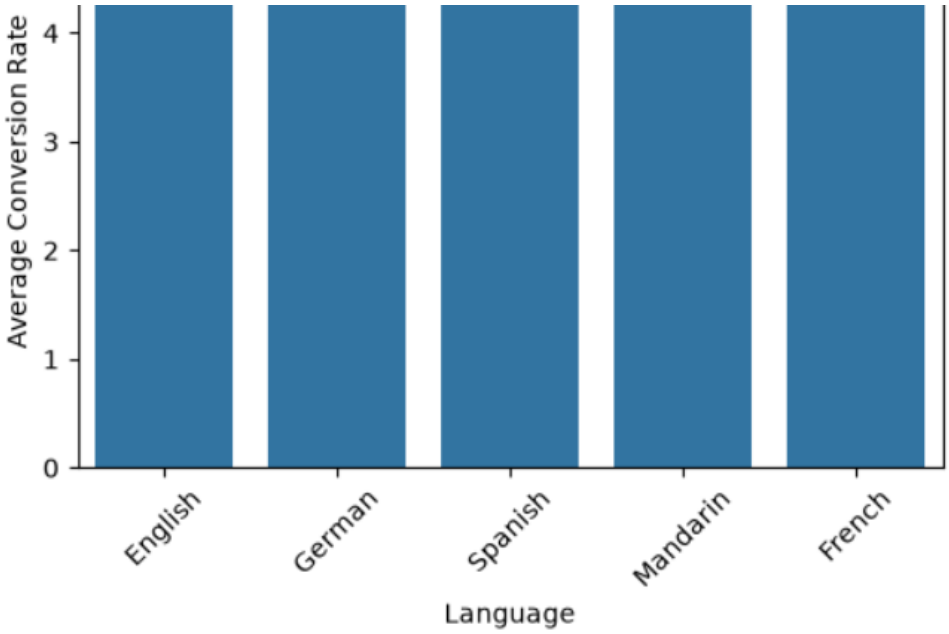
# Add title and labels
plt.title('Average Conversion Rate by Language')
plt.xlabel('Language')
plt.ylabel('Average Conversion Rate')

# Rotate labels if needed
plt.xticks(rotation=45)

# Display plot
plt.show()
```

Activate Windows
Go to Settings to activate Windows.





- A bar chart was used to compare the average **Conversion Rate** across different **Languages**.
- The average conversion rate for each language was calculated and visualized to identify which language achieved the highest campaign effectiveness.
- Languages with higher average conversion rates indicate better customer engagement and response to marketing campaigns, helping organizations optimize their multilingual marketing strategies.

Channel Effectiveness:

➢Compare the Engagement_Score for different Channels_Used, segmented by **Campaign_Type, using a bar chart.**

```
plt.figure(figsize=(6, 4))

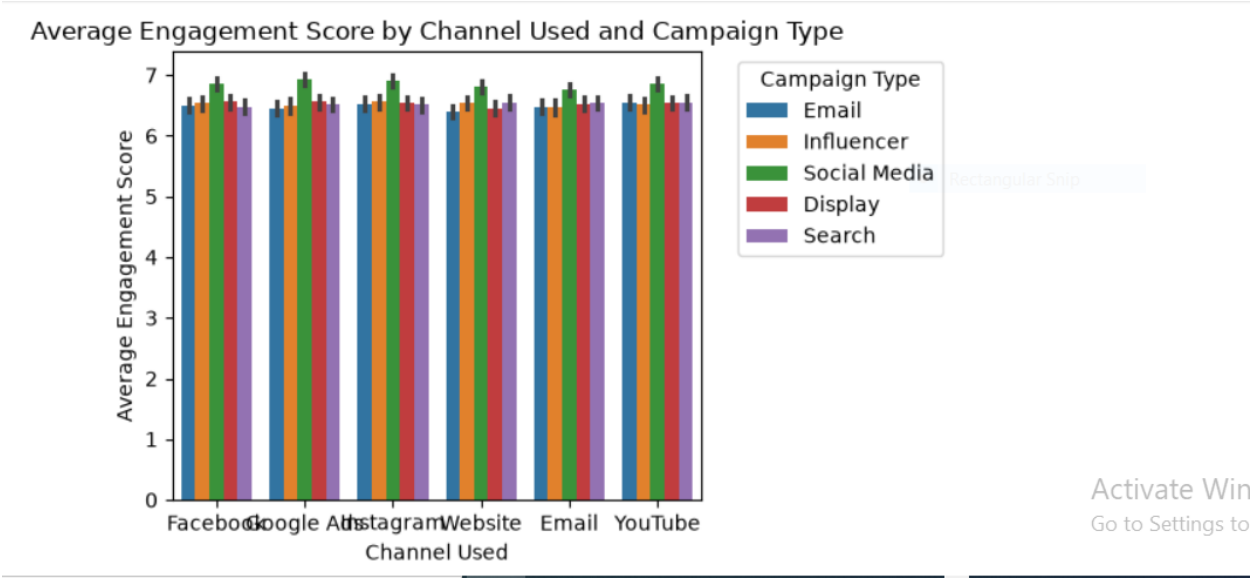
# Create grouped bar chart
sns.barplot(
    data=df,
    x='Channel_Used',
    y='Engagement_Score',
    hue='Campaign_Type',
    estimator=np.mean
)

# Add title and labels
plt.title('Average Engagement Score by Channel Used and Campaign Type')
plt.xlabel('Channel Used')
plt.ylabel('Average Engagement Score')

# Display legend
plt.legend(title='Campaign Type', bbox_to_anchor=(1.05, 1), loc='upper left')

# Show plot
plt.tight_layout()
plt.show()
```

Activate Windows
Go to Settings to activate Windows.



Activate Win
Go to Settings to

- A grouped bar chart was used to compare the average **Engagement_Score** across different **marketing channels** (`Channel_Used`) while segmenting the results by **Campaign_Type**.
- Each bar represents the mean engagement score for a specific combination of channel and campaign type.
- This visualization helps identify which channels generate the highest customer engagement and whether their effectiveness varies across different campaign strategies.

➢Show the distribution of total ROI across different Channels_Used using a pie chart.

```python
import matplotlib.pyplot as plt

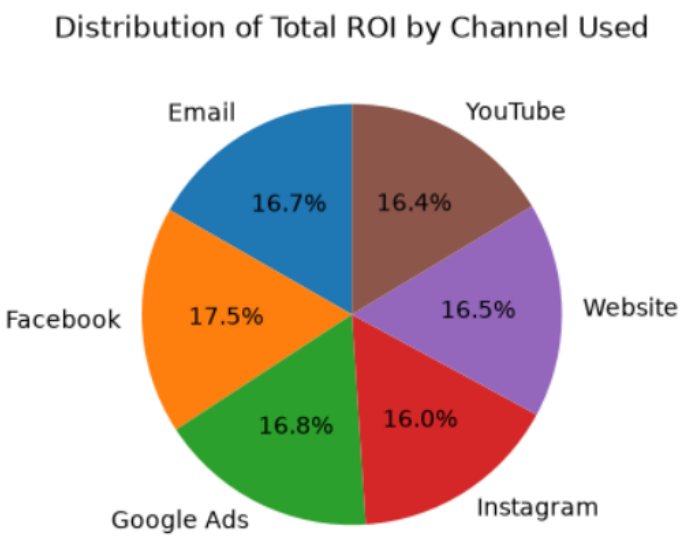
# Calculate total ROI by channel
roi_by_channel = df.groupby('Channel_Used')['ROI'].sum()

# Create pie chart
plt.figure(figsize=(4, 4))
plt.pie(
    roi_by_channel,
    labels=roi_by_channel.index,
    autopct='%1.1f%%',
    startangle=90
)

# Add title
plt.title('Distribution of Total ROI by Channel Used')

# Display chart
plt.show()
```

Activate Windows
Go to Settings to activate Windows.



Rectangular Snip

- A pie chart was used to visualize the distribution of **total ROI** across different **marketing channels** (`Channel_Used`).
- The ROI values were aggregated for each channel, and the resulting chart shows the percentage contribution of each channel to the overall ROI.
- This analysis helps identify which marketing channels generate the largest share of campaign returns and contribute most to overall profitability.

➢Plot a scatter plot to show the relationship between Clicks and Impressions **for each Channel_Used.**

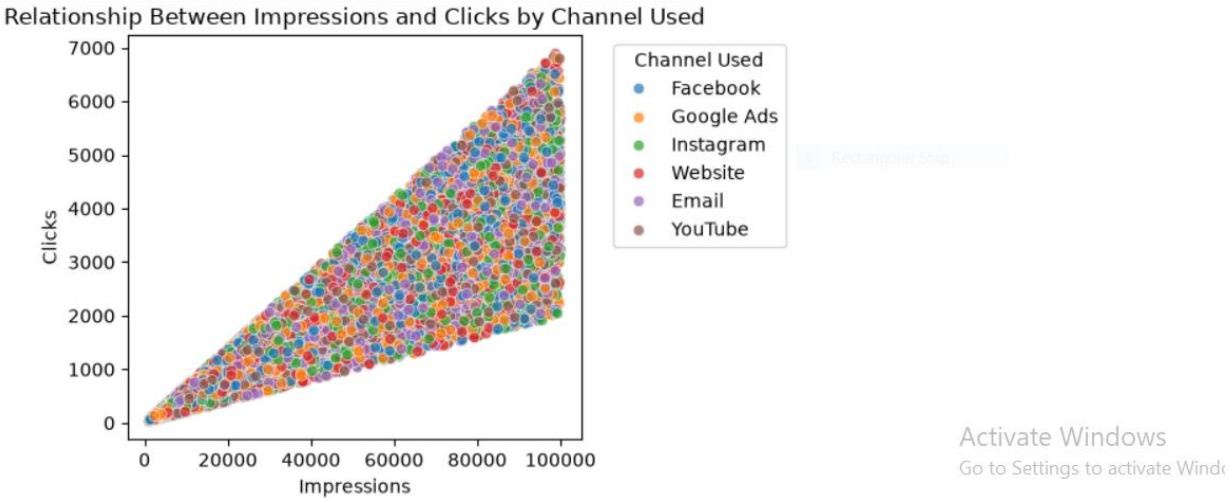
```python
# Create scatter plot
sns.scatterplot(
    data=df,
    x='Impressions',
    y='Clicks',
    hue='Channel_Used',
    alpha=0.7
)

# Add title and labels
plt.title('Relationship Between Impressions and Clicks by Channel Used')
plt.xlabel('Impressions')
plt.ylabel('Clicks')

# Display legend
plt.legend(title='Channel Used', bbox_to_anchor=(1.05, 1), loc='upper left')

# Show plot
plt.tight_layout()
plt.show()
```

Activate Windows
Go to Settings to activate Windows.



Activate Windows
Go to Settings to activate Windo

- A scatter plot was created to examine the relationship between **Impressions** and **Clicks** across different marketing channels (`Channel_Used`).
- Each point represents a campaign, and different colors indicate different channels.
- The plot helps identify whether campaigns with higher impressions tend to generate more clicks and whether this relationship varies across marketing channels.
- Trends, clusters, and outliers can also be observed, providing insights into channel effectiveness.

Time-Based Analysis:

➢Plot the distribution of Duration using a histogram.

```
import seaborn as sns

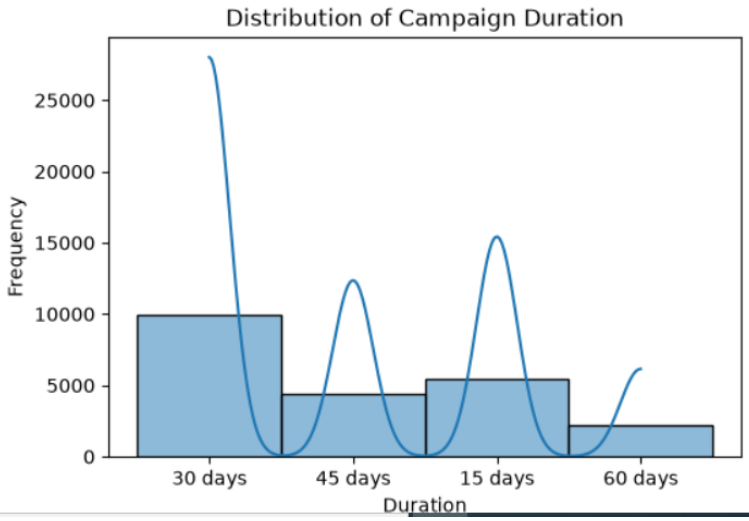
# Set figure size
plt.figure(figsize=(6, 4))

# Create histogram
sns.histplot(
    data=df,
    x='Duration',
    bins=20,
    kde=True
)

# Add title and labels
plt.title('Distribution of Campaign Duration')
plt.xlabel('Duration')
plt.ylabel('Frequency')

# Display plot
plt.show()
```

Activate Windows
Go to Settings to activate Windows.



Activate Windows
Go to Settings to activat

- A histogram was used to visualize the distribution of the **Duration** variable.
- The chart shows how frequently different campaign durations occur within the dataset.
- The optional KDE (Kernel Density Estimation) curve helps identify the overall distribution pattern, such as whether campaign durations are normally distributed, skewed, or contain multiple peaks.
- This analysis provides insight into the typical length of marketing campaigns and the spread of campaign durations.

➢Analyze how the overall Conversion_Rate has changed over Date for each Company using a line chart.

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Convert Date column to datetime format
df['Date'] = pd.to_datetime(df['Date'] , dayfirst=True)

# Calculate average Conversion Rate by Date and Company
conversion_trend = df.groupby(['Date', 'Company'])['Conversion_Rate'].mean().reset_index()

# Create line chart
plt.figure(figsize=(10, 5))

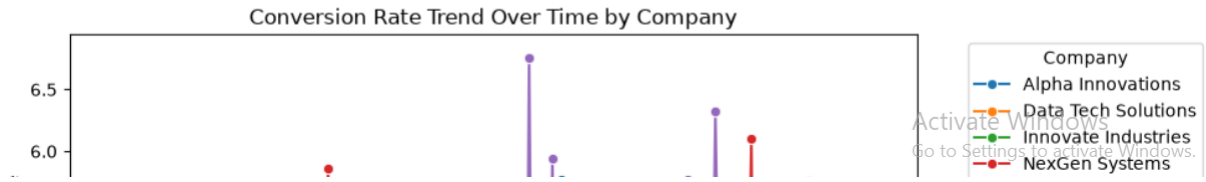
sns.lineplot(
    data=conversion_trend,
    x='Date',
    y='Conversion_Rate',
    hue='Company',
    marker='o'
)
```

Activate Windows
Go to Settings to activate Windows.

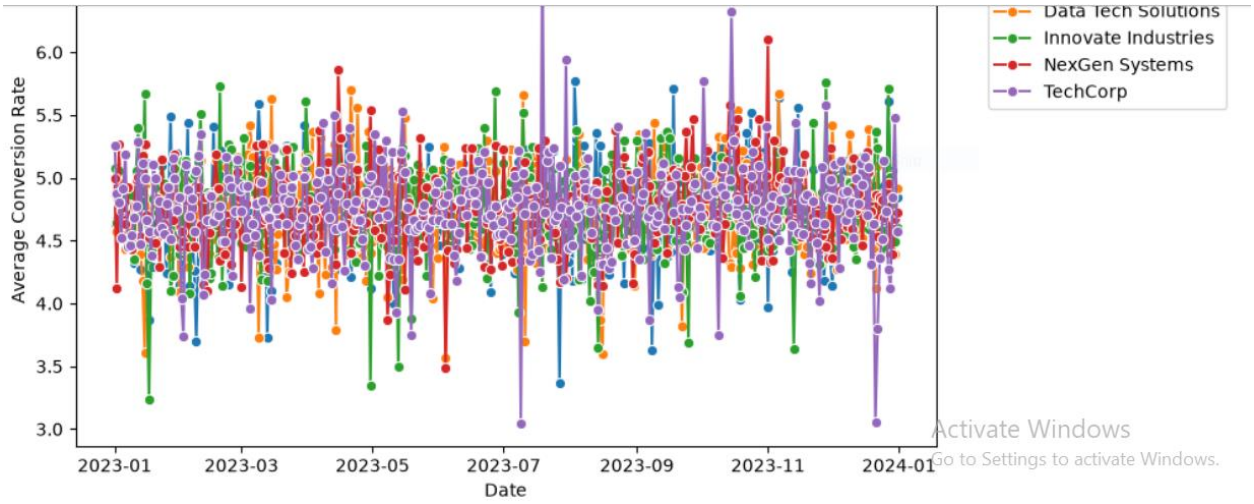
```python
# Add title and labels
plt.title('Conversion Rate Trend Over Time by Company')
plt.xlabel('Date')
plt.ylabel('Average Conversion Rate')

# Display legend
plt.legend(title='Company', bbox_to_anchor=(1.05, 1), loc='upper left')

# Improve layout
plt.tight_layout()
plt.show()
```



Activate Windows
Go to Settings to activate Windows.



- A line chart was used to analyze changes in the **Conversion Rate** over time for different companies.
- The data was grouped by **Date** and **Company**, and the average conversion rate was calculated for each date.
- Each line represents a company, allowing comparison of campaign performance trends over time.
- This visualization helps identify periods of improvement or decline in conversion rates and highlights companies with consistently strong performance.

➢Examine the trend of Engagement_Score over Date with a line chart.

```python
# the trend of Engagement_Score over Date with a line chart.
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Convert Date column to datetime
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')

# Calculate average Engagement Score by Date
engagement_trend = df.groupby('Date')['Engagement_Score'].mean().reset_index()

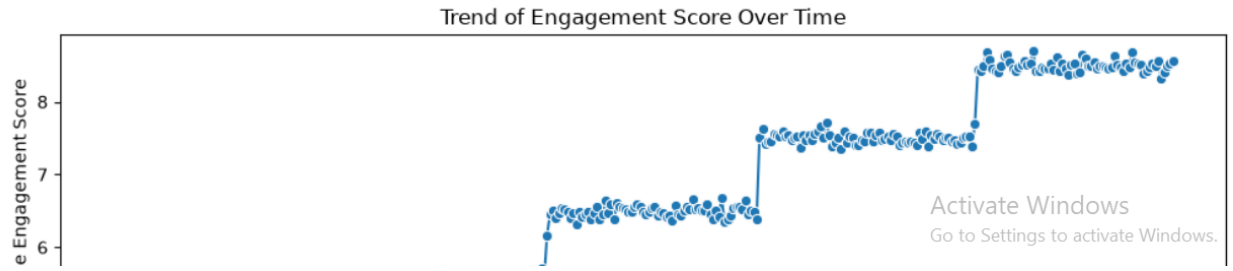
# Create line chart
plt.figure(figsize=(10, 4))

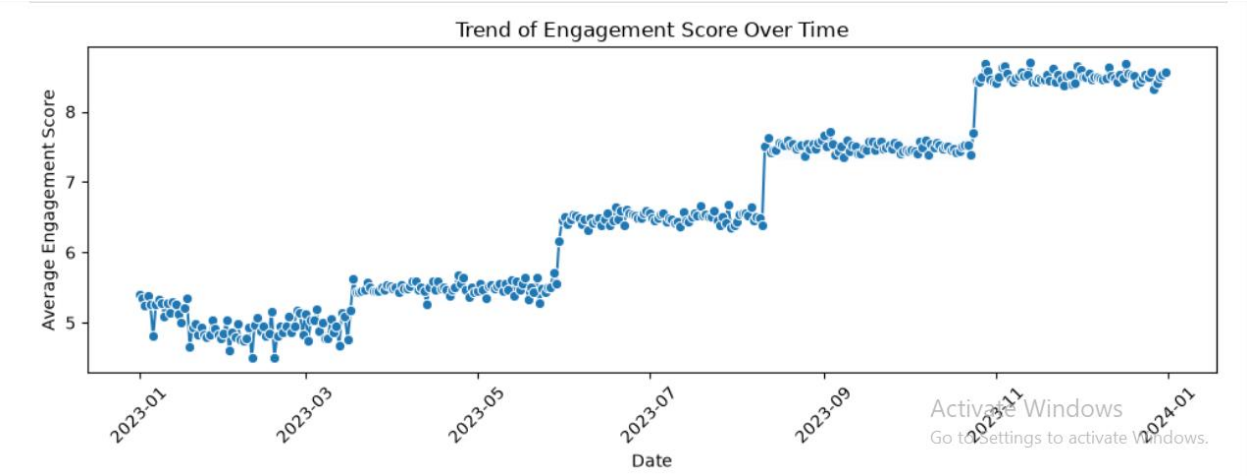
sns.lineplot(
    data=engagement_trend,
    x='Date',
    y='Engagement_Score',
    marker='o'
)
```

Activate Windows
Go to Settings to activate Windows.

```
# Add title and labels
plt.title('Trend of Engagement Score Over Time')
plt.xlabel('Date')
plt.ylabel('Average Engagement Score')

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```





- A line chart was used to analyze the trend of **Engagement Score** over time.
- The average engagement score was calculated for each date and plotted to observe changes in customer engagement throughout the campaign period.
- This visualization helps identify periods of high or low engagement and reveals overall engagement patterns over time.

Geographic Analysis:

➢Determine which location has the highest Acquisition_Cost using a bar chart.

```python
# Geographic Analysis:
# the highest Acquisition_Cost using a bar chart.
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate average acquisition cost by location
location_cost = df.groupby('Location')['Acquisition_Cost'].mean().reset_index()

# Sort values for better visualization
location_cost = location_cost.sort_values('Acquisition_Cost', ascending=False)

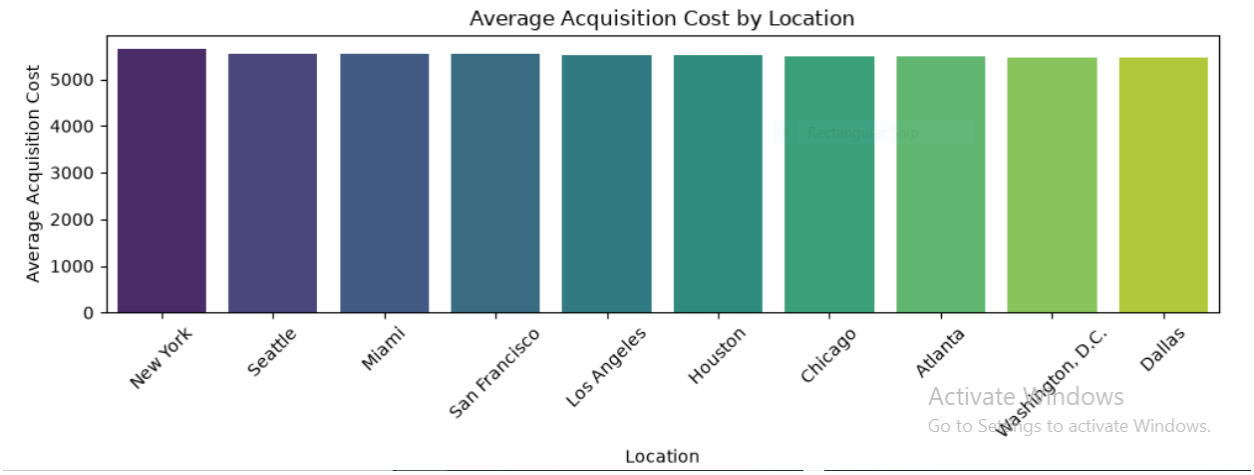
# Create bar chart
plt.figure(figsize=(10, 4))

sns.barplot(
    data=location_cost,
    x='Location',
    y='Acquisition_Cost',
    palette='viridis'
)
```

Activate Windows
Go to Settings to activate Windows.

```python
# Add title and labels
plt.title('Average Acquisition Cost by Location')
plt.xlabel('Location')
plt.ylabel('Average Acquisition Cost')

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Activate Windows
Go to Settings to activate Windows.

- A bar chart was used to compare the average **Acquisition Cost** across different locations.
- The data was grouped by **Location**, and the mean acquisition cost was calculated for each location.

- The chart helps identify which geographic region requires the highest customer acquisition investment, providing insights into regional marketing expenses and resource allocation.

➢Visualize the Conversion_Rate by different Location, categorized by Target_Audience, using a bar chart.

```
# the Conversion_Rate by different Location, categorized by
# Target_Audience, using a bar chart.
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Set figure size
plt.figure(figsize=(10, 4))

# Create bar chart
sns.barplot(
    data=df,
    x='Location',
    y='Conversion_Rate',
    hue='Target_Audience',
    estimator=np.mean
)
```

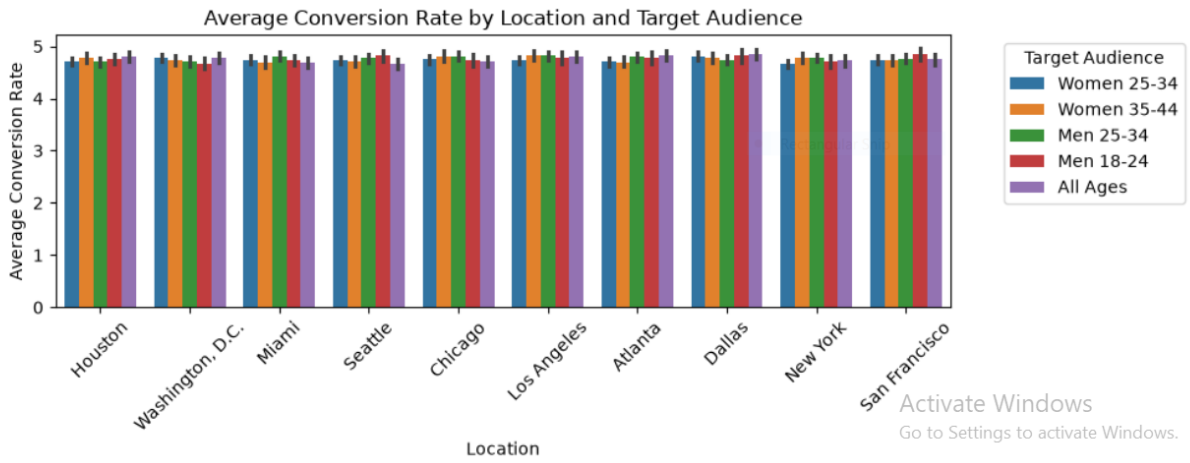
```
# Add title and labels
plt.title('Average Conversion Rate by Location and Target Audience')
plt.xlabel('Location')
plt.ylabel('Average Conversion Rate')

# Rotate x-axis labels
plt.xticks(rotation=45)

# Show legend
plt.legend(title='Target Audience', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.show()
```

Activate Windows
Go to Settings to activate Windows.



Activate Windows
Go to Settings to activate Windows.

- A grouped bar chart was created to compare the average **Conversion Rate** across different **Locations**, segmented by **Target Audience**.
- The chart helps identify which audience groups perform best in each location and reveals geographic differences in campaign effectiveness.
- Higher bars indicate stronger conversion performance for a specific target audience within a location.

➢Illustrate the proportion of ROI by Location using a pie chart.

```python
# the proportion of ROI by Location using a pie chart.
import matplotlib.pyplot as plt

# Calculate total ROI by Location
roi_by_location = df.groupby('Location')['ROI'].sum()

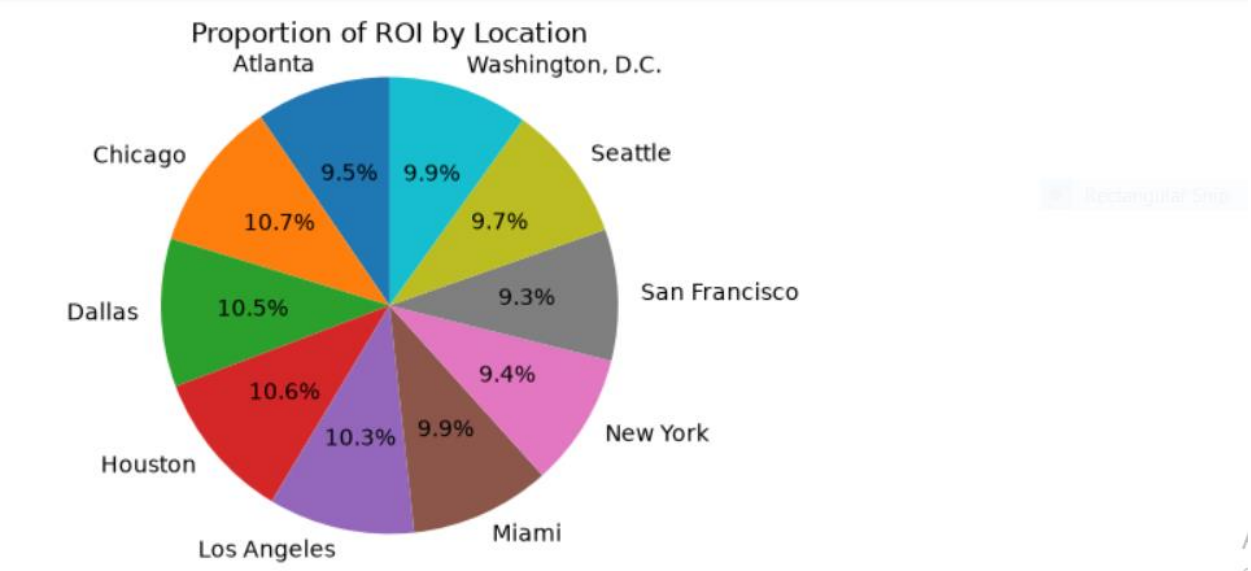
# Create pie chart
plt.figure(figsize=(4, 4))

plt.pie(
    roi_by_location,
    labels=roi_by_location.index,
    autopct='%1.1f%%',
    startangle=90
)

# Add title
plt.title('Proportion of ROI by Location')

# Make pie chart circular
plt.axis('equal')
```

Activate Windows
Go to Settings to activate Windows.



- A pie chart was used to visualize the proportion of **total ROI** contributed by each **Location**.
- The ROI values were aggregated for each location, and the resulting chart displays the percentage contribution of each region to the overall return on investment.
- This analysis helps identify the locations that generate the greatest share of campaign profitability and highlights geographic variations in marketing performance.